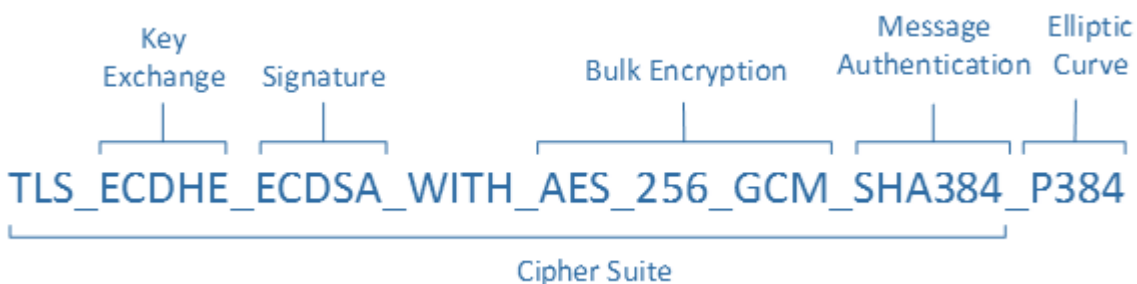


SSL and TLS:

- o SSL stands for **Secure Socket Layer** TLS **Transport Layer Security**.
- o Both protocols provide security between web server & web browser.
- o Both SSL and TLS are protocols, which serve the same purpose.
- o However, there are minor differences between SSL and TLS.
- o SSL offers two basic security services: Authentication and confidentiality.
- o Provides secure connection between the web browser and the web server.
- o Both SSL & TLS provide encryption and authentication of data in motion.
- o Common applications of such protocols are web browsing, VOIP & electronic mail.
- o SSL was developed by Netscape in 1994 with an intention to protect web transactions.
- o Transport Layer Security (TLS) is IETF (Internet Engineering Task Force) standardization.
- o Secure Socket Layer (SSL) and TLS both Protection of normal TCP/UDP connection.
- o SSL/TLS can also be used for securing other protocols like FTP, SMTP and SNTP etc.
- o TLS uses stronger encryption algorithms & has the ability to work on different ports.
- o SSL works mainly through using public and private key encryption on data.
- o The TLS has two layers of operations while it establishes the communication.
- o First, one is Handshaking to authenticate server & second is actual message transfer.
- o TLS Takes a little more time than the older SSL to establish connections and transfers.
- o SSL, or Secure Sockets Layer, was the original encryption protocol developed for HTTP.
- o SSL was replaced by TLS, some time ago; SSL handshakes are now called TLS handshake.
- o SSL is the combination of Handshake, Change Cipher Spec, Alert and Record Protocols.

Protocol	Published	Status
SSL 1.0	Unpublished	Unpublished only for their internal testing
SSL 2.0	1995	Deprecated in 2011
SSL 3.0	1996	Deprecated in 2015
TLS 1.0	1999	Deprecation planned in 2020
TLS 1.1	2006	Deprecation planned in 2020
TLS 1.2	2008	
TLS 1.3	2018	





Website Identity

Website: security.stackexchange.com
Owner: This website does not supply ownership information.
Verified by: Let's Encrypt
Expires on: Thursday, October 24, 2019

[View Certificate](#)

Privacy & History

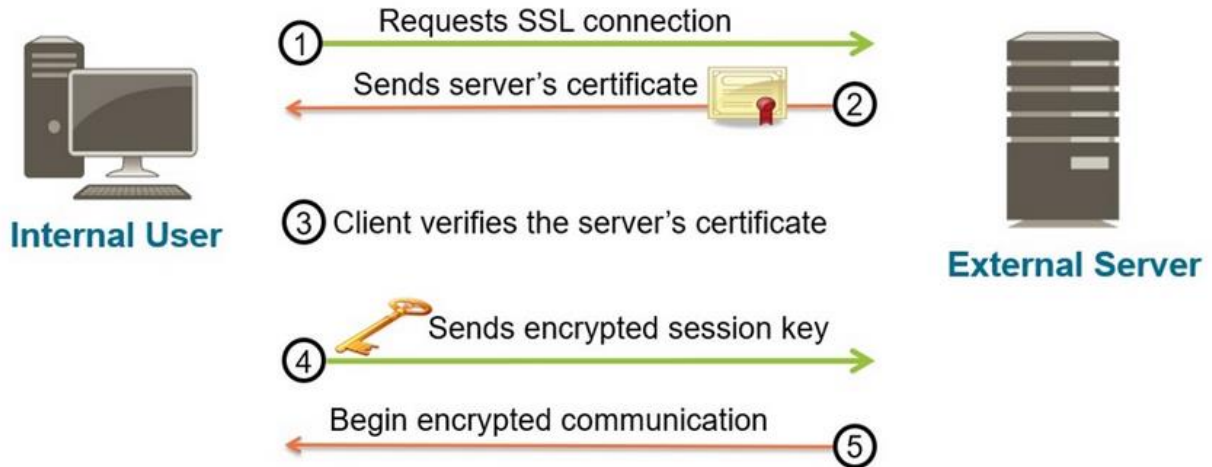
Have I visited this website prior to today? Yes, 23 times
Is this website storing information on my computer? Yes, cookies
Have I saved any passwords for this website? No

[Clear Cookies and Site Data](#)

[View Saved Passwords](#)

Technical Details

Connection Encrypted (TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256, 128 bit keys, TLS 1.2)
The page you are viewing was encrypted before being transmitted over the Internet.
Encryption makes it difficult for unauthorized people to view information traveling between computers. It is therefore unlikely that anyone read this page as it traveled across the network.



SSL/TLS Handshake Explained:

Step1. Client Hello:

The client begins the communication. The first step is called **client hello**. The client lists **the versions of SSL/TLS** and **cipher suites** it's able to use.

```
- Secure Sockets Layer
  TLSv1.2 Record Layer: Handshake Protocol: Client Hello
    Content Type: Handshake (22)
    Version: TLS 1.0 (0x0301)
    Length: 186
  Handshake Protocol: Client Hello
    Handshake Type: Client Hello (1)
    Length: 182
    Version: TLS 1.2 (0x0303)
  Random
    Session ID Length: 0
    Cipher Suites Length: 22
  Cipher Suites (11 suites)
    Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 (0xc02b)
    Cipher Suite: TLS_ECDHE_RSA_WITH_AES_128_GCM_SHA256 (0xc02f)
    Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_256_CBC_SHA (0xc00a)
    Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_128_CBC_SHA (0xc009)
    Cipher Suite: TLS_ECDHE_RSA_WITH_AES_128_CBC_SHA (0xc013)
    Cipher Suite: TLS_ECDHE_RSA_WITH_AES_256_CBC_SHA (0xc014)
    Cipher Suite: TLS_DHE_RSA_WITH_AES_128_CBC_SHA (0x0033)
    Cipher Suite: TLS_DHE_RSA_WITH_AES_256_CBC_SHA (0x0039)
    Cipher Suite: TLS_RSA_WITH_AES_128_CBC_SHA (0x002f)
    Cipher Suite: TLS_RSA_WITH_AES_256_CBC_SHA (0x0035)
    Cipher Suite: TLS_RSA_WITH_3DES_EDE_CBC_SHA (0x000a)
    Compression Methods Length: 1
```

SSL/TLS versions supported by client

Cipher Suites supported by client

Step2. Server Hello:

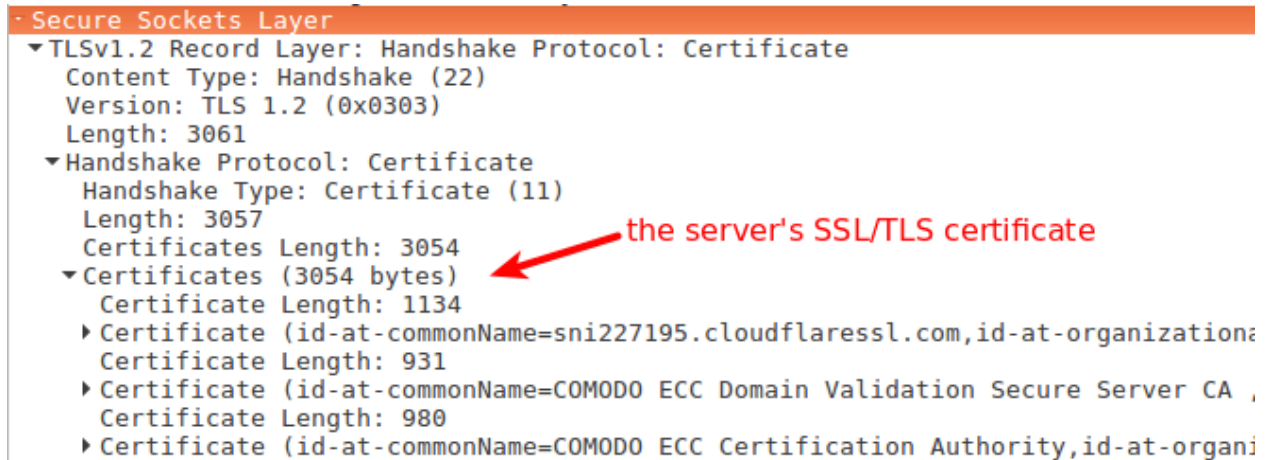
The server will see the list of SSL/TLS versions and cipher suites and pick the newest the server is able to use. Then the server sends a message to the client containing the SSL/TLS version and cipher suite it chose.

```
- Secure Sockets Layer
  TLSv1.2 Record Layer: Handshake Protocol: Server Hello
    Content Type: Handshake (22)
    Version: TLS 1.2 (0x0303)
    Length: 70
  Handshake Protocol: Server Hello
    Handshake Type: Server Hello (2)
    Length: 66
    Version: TLS 1.2 (0x0303)
  Random
    Session ID Length: 0
    Cipher Suite: TLS_ECDHE_ECDSA_WITH_AES_128_GCM_SHA256 (0xc02b)
    Compression Method: null (0)
    Extensions Length: 26
  Extension: server_name
  Extension: renegotiation_info
  Extension: ec_point_formats
  Extension: Application Layer Protocol Negotiation
```

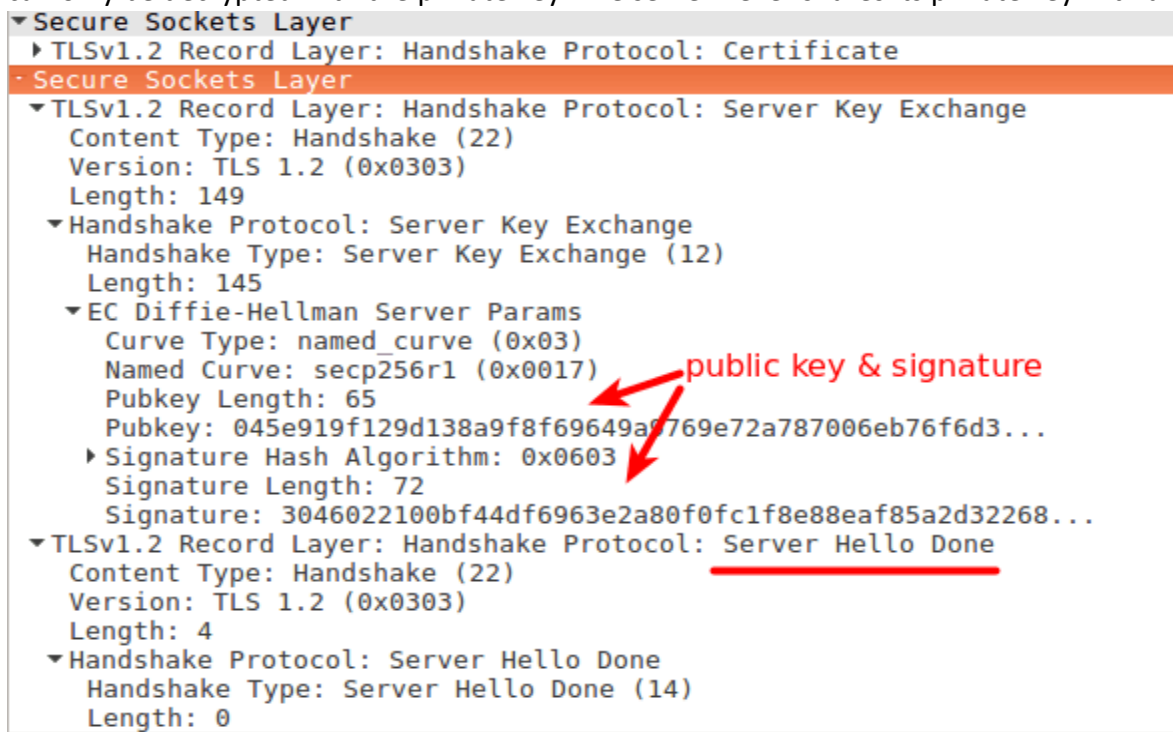
SSL/TLS version and cipher suite picked by the server

Step3. Server Key Exchange:

After the server and client agree on the SSL/TLS version and cipher suite, then server sends two things. The first is its **SSL/TLS certificate** to the client. The client (web browser) validates the server's certificate. Web browsers store a list of Root CA(Certificate Authority) in itself. These root CAs are third parties that are trusted by web browsers. The server's certificate is issued by root CA or immediate CA. Immediate CA is a CA that is trusted by root CA. Web browsers trust Root CA. Root CA trust immediate CA. If the server's certificate is issued by a trusted root CA or immediate CA, then the browser trusts the server's certificate.



The second thing the server sends is its **public key and signature**. The public key is actually included in the certificate. The client and the server encrypt message with the public key, and it can only be decrypted with the private key. The server never shares its private key with anyone.



At the end of server key exchange, the server sends a **server hello done** message.

Step4. Client Key Exchange:

Until now, all the information sent between the client and server is unencrypted. Now the client receives the server's public key and generate a new session key (aka pre-master key) encrypted with the public key and sends it to the server. The session key can only be decrypted with the private key and because only the server has the private key so only the client and server know the session key. This session key is only valid in one session. If the user closes the client and visit the same server next day, a new session key will be generated by the client.

```
▼ Secure Sockets Layer
  ▼ TLSv1.2 Record Layer: Handshake Protocol: Client Key Exchange
    Content Type: Handshake (22)
    Version: TLS 1.2 (0x0303)
    Length: 70
  ▼ Handshake Protocol: Client Key Exchange
    Handshake Type: Client Key Exchange (16)
    Length: 66
  ▼ EC Diffie-Hellman Client Params
    Pubkey Length: 65
    Pubkey: 04126586c33ff7fcde4250ec9d289b2fda25fc0c9f3090f6...
```

This pubkey is actually the session key.

Step 5. Change Cipher Spec:

The change cipher spec message is sent by both the client and server to notify the receiving party that subsequent records will be protected under the just-negotiated CipherSpec and keys.

Step 6. Encrypted Handshake:

The client and the server send to each other an encrypted message saying the key information is correct. Now the client (web browser) will see a green lock in the address bar. The client and server encrypt http traffic with the session key.



```
▼ Transport Layer Security
  ▼ TLSv1.3 Record Layer: Application Data Protocol: http-over-tls
    Opaque Type: Application Data (23)
    Version: TLS 1.2 (0x0303)
    Length: 182
    Encrypted Application Data: b2dd518dfbd75f53296d76c57aac8d62f2e40b2ed04afd1c...
```